

Daniel S

Professor Lewis

Asm6 Write Up

29 April 2026

Asm6 - Chrome Dinosaur Demo

My asm6 program is a tiny toy example of the Chrome offline dinosaur game. This program utilizes both the graphics and the keyboard inputs. My version is a bare bones variation of the dinosaur game. It only has a jump for the character and one moving obstacle that increases in speed as you go. My code is broken up into numerous functions to make the program modular.

The main function is the longest since it incorporates the game loop. The game loop runs infinitely and first checks if the player has put in a key stroke of the space bar. If so then it updates the player's character which is a white dot to move up one pixel. Next it then moves the enemy which is a green dot from right to left at the very bottom of the screen. The green dot moves three times per frame compared to the player which allows for the enemy to move faster. After the green dot movement the program checks if the player jumped. If so, then the program puts the player back on the ground.

If the player has survived the jump and did not collide with the enemy then their score is updated (+1) and their total points are then printed out to the terminal. The game continues and when the green dot loops back around and starts moving again from right to left, the frame speed for the game is increased by 10 milliseconds. Each time the player survives, the frames time shortens so it makes the green dot move faster but also makes the timing of moving the player a bit more tricky.

If the player does not survive the jump by colliding with the green dot then the game ends. Once the player collides with the green dot then an end screen plays which fills up the screen line by line with red from bottom to the top. Then stats about the player are printed to the terminal with the total amount of points, jumps and a “Better luck next time” message is printed. Once all of that is finished, the program is terminated.

How the player moves is very simple. At the beginning of the game loop the program checks if the player pressed the space bar. If so then the program calls the move dot function which then moves the player’s dot up one pixel. Once the program is ready for the player to stop being in the air then the same function is called but moves the player back down. This is done by indexing memory and changing the correct slots to white then back to black when they are not used.

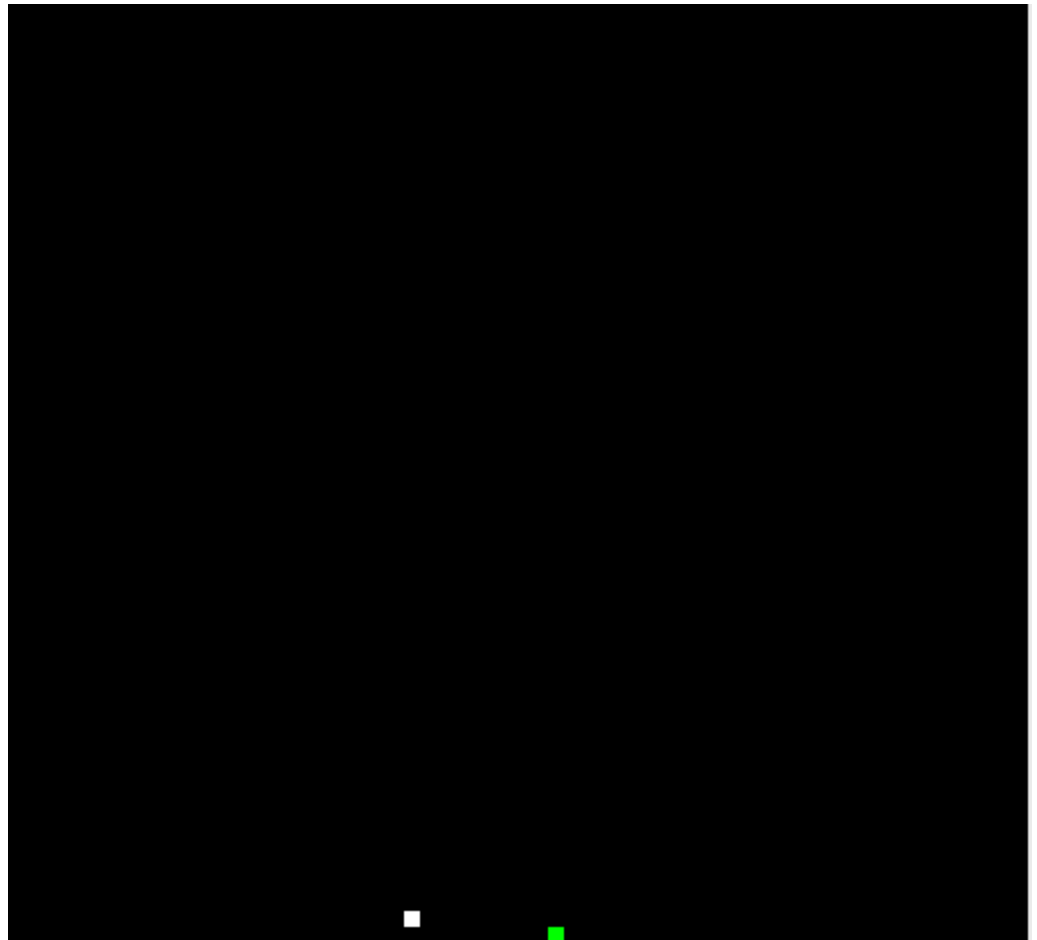
The enemy is moved in a similar manner to the player. The enemy starts at the bottom right corner of the screen and moves one pixel at a time to the left. To do the enemy movement there is a separate function that puts a black pixel where the current place the green dot is in then puts a green dot one step ahead. This makes it look like the green dot is moving. Since the program puts a black box behind where it is going that means when it starts in the lower right hand corner there is a bit of a spill over in memory. This is because a black box is put one word outside the chunk of memory dedicated to the screen when it starts at the bottom right hand corner. To deal with that in the .data section we have a one word buffer to easily fix that.

The program detects collision through checking when the player is not jumping and when the green dot is in the column that the player is at. If the player's jumping flag is not set to true and the green dot is in the column the player is in then they collide thus the ending program functions are called.

When the collision is detected then the first function that is called is the end screen function. This function loops through memory and turns one row at a time to all red. This loop is buffered for each row by a few milliseconds. The end screen then looks like it is filling up with red from the bottom to the top. Then the next function that is called is the print stats function which then prints the players stats and lastly the program is terminated.

Here are some images of the program visuals and terminal outputs:

This image is when the player is in the air in the 'jumping' state.



Here is an image of the end screen in the process of filling up the screen with red.



This image is a terminal output that shows the points the player gets while playing the game and once the player collides with the green dot then it prints out the total points and jumps with an encouraging message.

```
POINTS: 1  
POINTS: 2  
TOTAL POINTS: 2  
TOTAL JUMPS: 3  
Better luck next time.
```